Spell Idel

장태훈

INDEX

목차

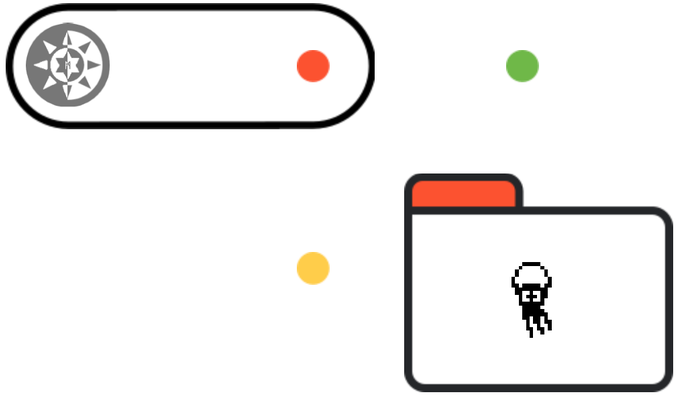
1 게임 개요

2 게임 설명

3 주요 구현 기술

01

게임 개요



1. 게임 개요

게임 이름 : **Spell Idel**

장르 : 캐주얼 머지 방치형게임

개발환경 : Unity 2022.3.4f1 / C#

개발기간 : 2026년 1월12일 ~ 2월18일 (40일)

게임 소개 : 마법진(스펠)을 소환하고, 머지하여 마법진을 장착하면 자동으로 몬스터를 공격해 스테이지를 클리어하고. 보스를 사냥하는 방치형 머지게임입니다.

Github : ttps://github.com/BambaStone/MagicCircle_Idle

빌드파일 :

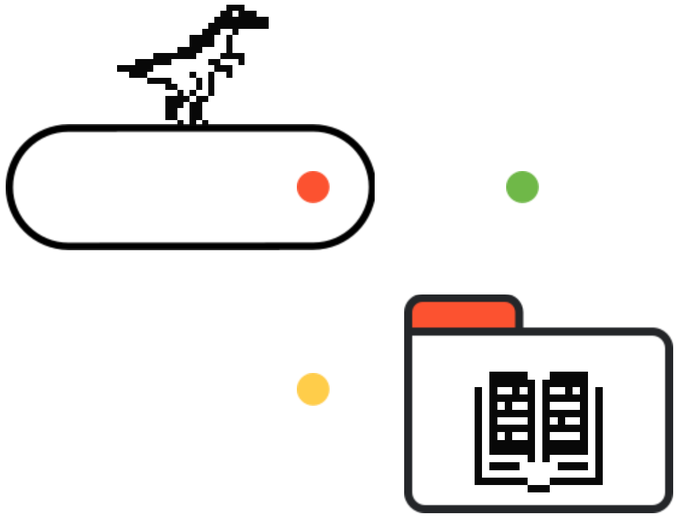
apk파일 : https://drive.google.com/file/d/1AdSA-RlNJOIS4mnA4V-WnC_NWhvFz5o3/view?usp=drive_link

PC_exe파일 : https://drive.google.com/file/d/1fZrvhUDs3FC-jjJIw3MYiWcU2KkPhU3O/view?usp=drive_link

플레이 영상 : https://youtu.be/sJ9Kwf-DNGU?si=QS0akNLIuhglOQXz

02

게임 설명



게임의 핵심 규칙

1단계 Make

스펠을 소환

2단계 Collect

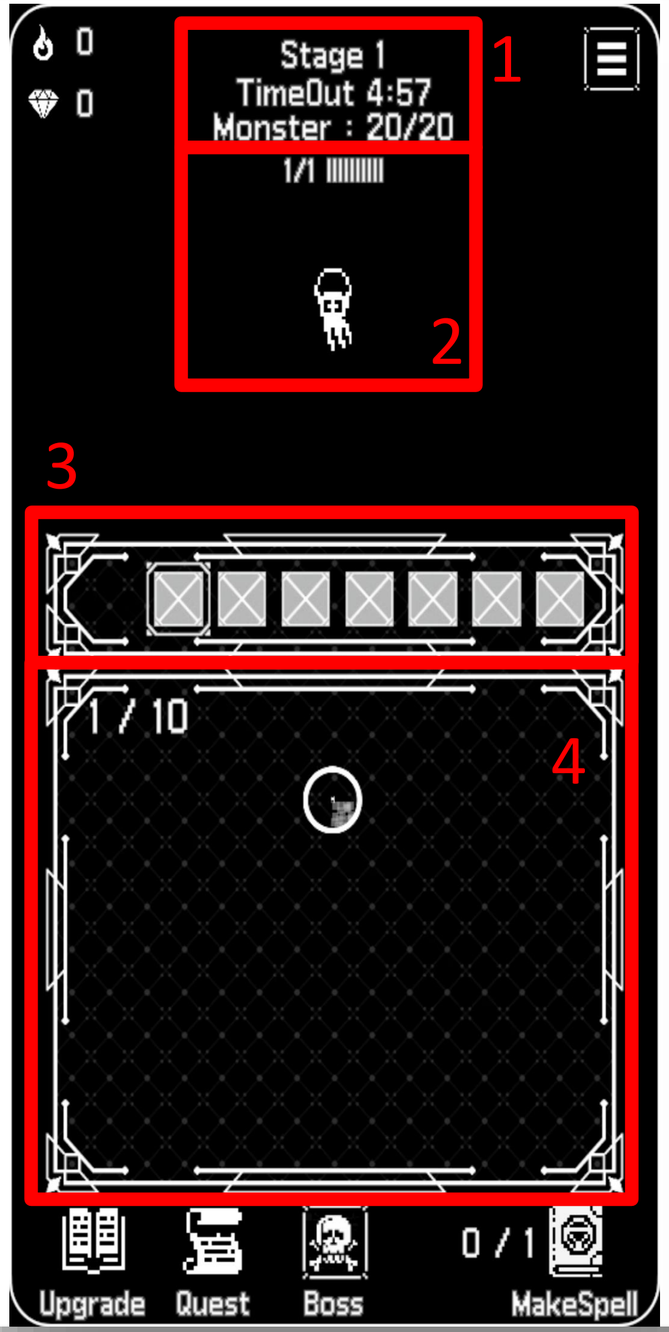
몬스터를 사냥해서 재화를 수집

3단계 Upgrade

소환한 스펠을 합성, 또는 수집한 재화로 강화

4단계 Progress

강화된 스펠로 더 강한 스테이지와 보스를 클리어해서 다시 재화를 수집



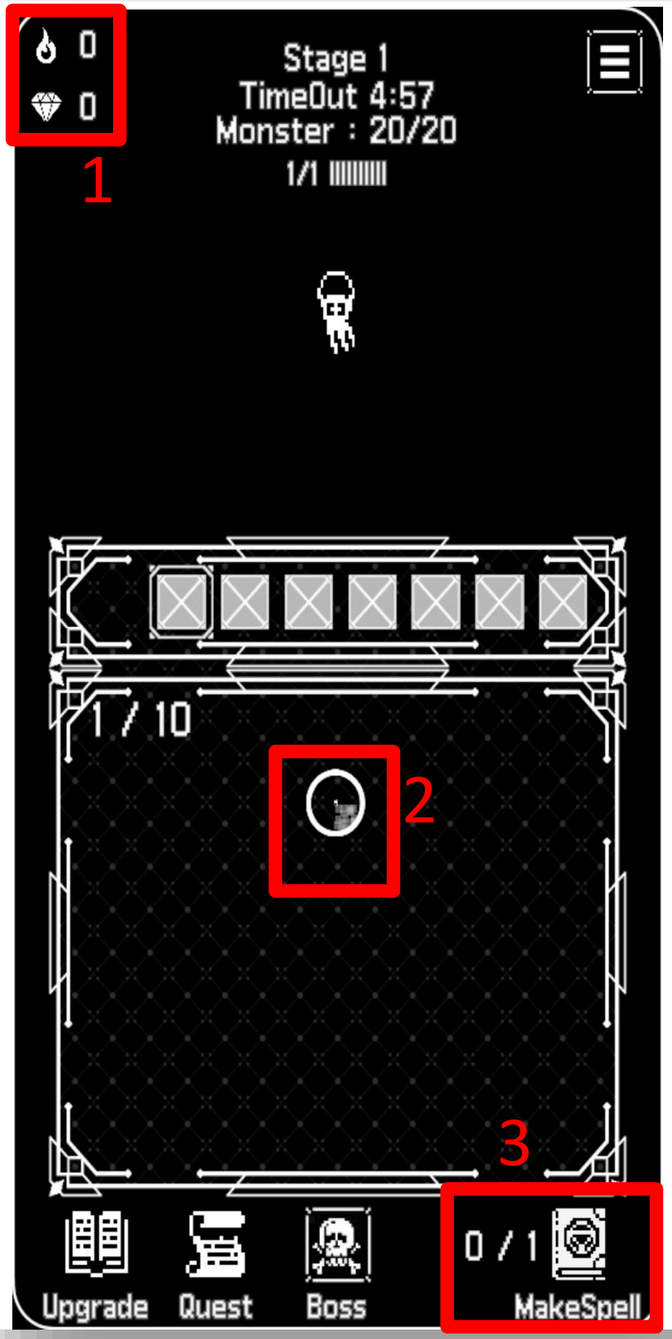
UI 설명 - 메인화면

1. 스테이지 설명
   현재 스테이지, 스테이지 클리어를 위한 시간 제한, 스테이지의 남은 몬스터 수를 표시해줍니다.
   시간 제한이 0이되면 해당 스테이지를 다시 시작합니다.
   스테이지의 남은 몬스터가 0이되면 다음 스테이지로 바로 진입합니다.

2. 몬스터
   현재 상대중인 몬스터입니다.
   위쪽에 체력이 표시되어있고, 아래쪽이 몬스터의 모습을 표시해줍니다.
   몬스터의 HP가 0이하가 되면 다음 몬스터가 등장합니다.

3. 스펠발사대
   소환한 스펠을 장착해 몬스터를 향해 발동시키는 공간입니다.
   스펠을 드래그해서 장착할 수 있습니다.
   X자 박스는 해당 공간이 잠겨있어 업그레이드로 해금할수 있습니다.

4. 스펠저장소
   스펠을 소환하면 보관되는 공간입니다.
   왼쪽 위에 현재 소유한 스펠수와 최대 소유 가능한 스펠 수가 표시됩니다.
   업그레이드로 최대 소유 가능한 스펠 수를 늘릴 수 있습니다.

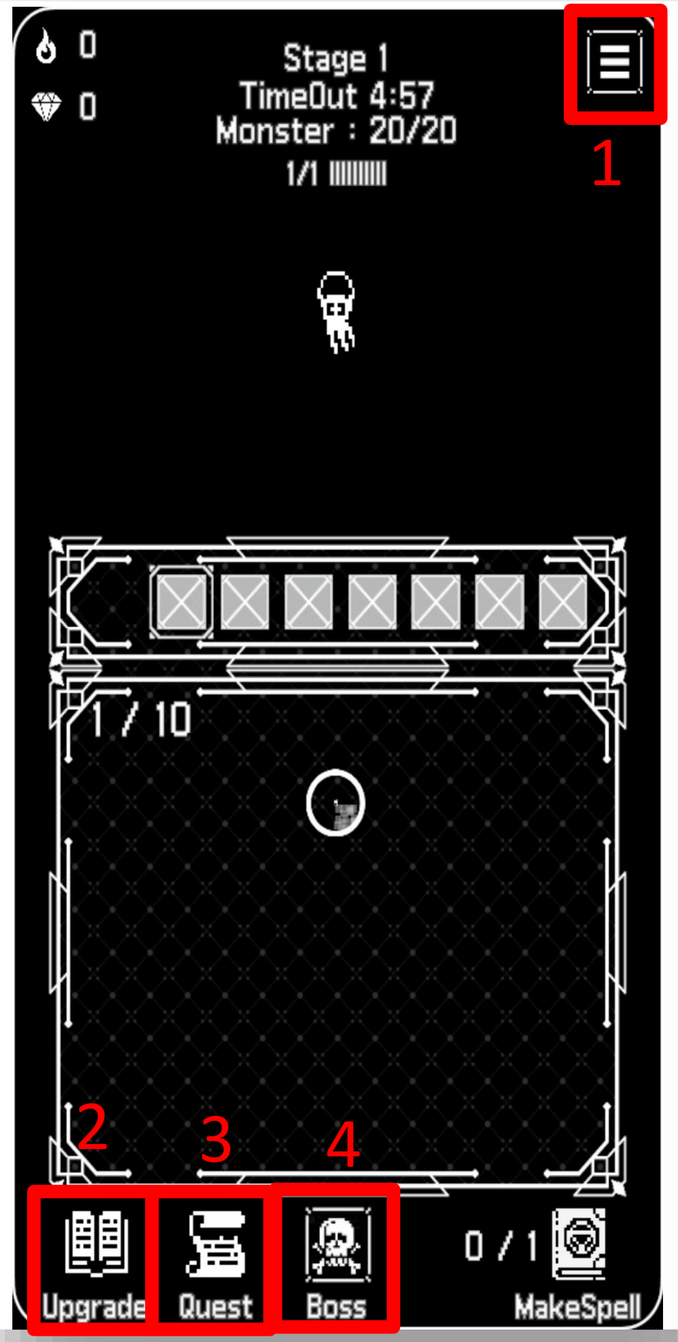


UI 설명 - 메인화면

1. 재화
   소유중인 재화를 표시해줍니다.
   몬스터를 사냥하거나 보스를 사냥하면 재화가 추가됩니다.
   1분마다 스테이지 클리어, 보스 클리어 에 맞춰서 환산한 방치재화를 추가해줍니다.

2. 스펠
   소환한 스펠입니다.
   같은 단계의 스펠을 드래그로 합치면 다음단계의 스펠이 소환됩니다.

3. MakeSpell버튼
   누르면 스펠을 소환합니다.
   현재 소환 가능 횟수와 최대 소유 가능한 소환 횟수가 표시됩니다.
   1분에 1씩 소환 가능 횟수가 회복됩니다.



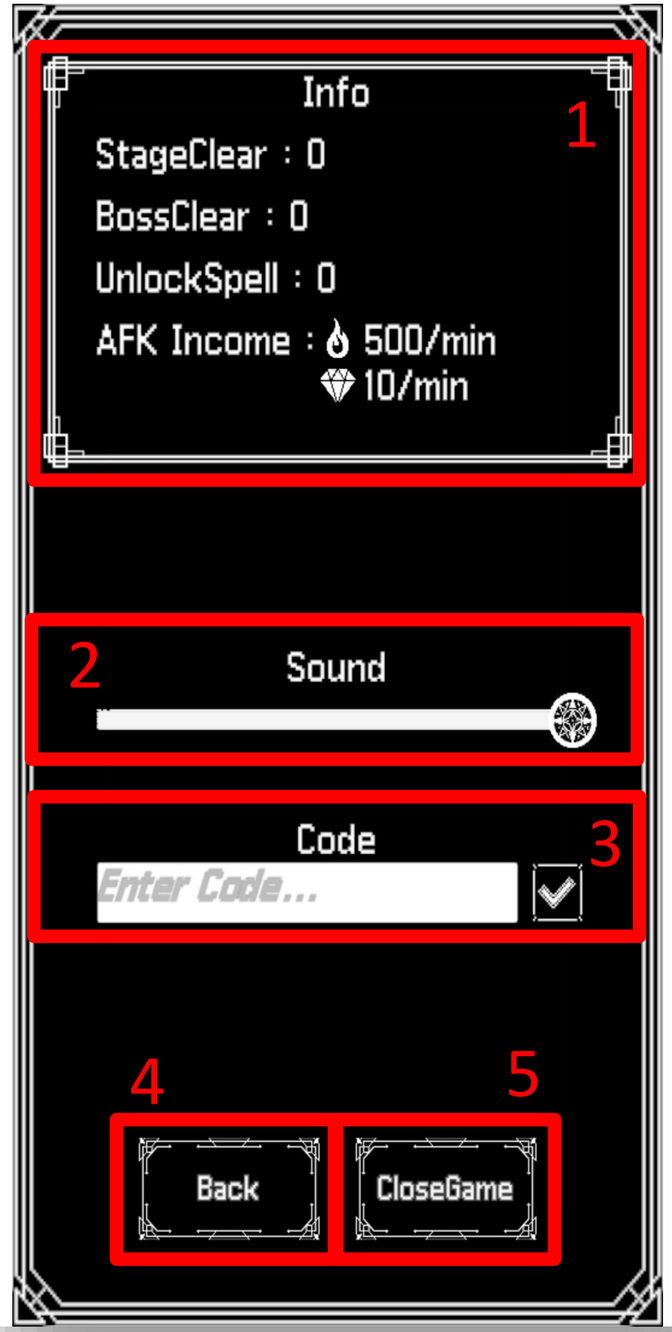
UI 설명 - 메인화면

1. 메뉴버튼
   메뉴UI를 오픈합니다.

2. 업그레이드 버튼
   업그레이드UI를 오픈합니다.

3. 퀘스트 버튼
   퀘스트UI를 오픈합니다.

4. 보스 버튼
   보스UI를 오픈합니다.



UI 설명 - 메뉴UI

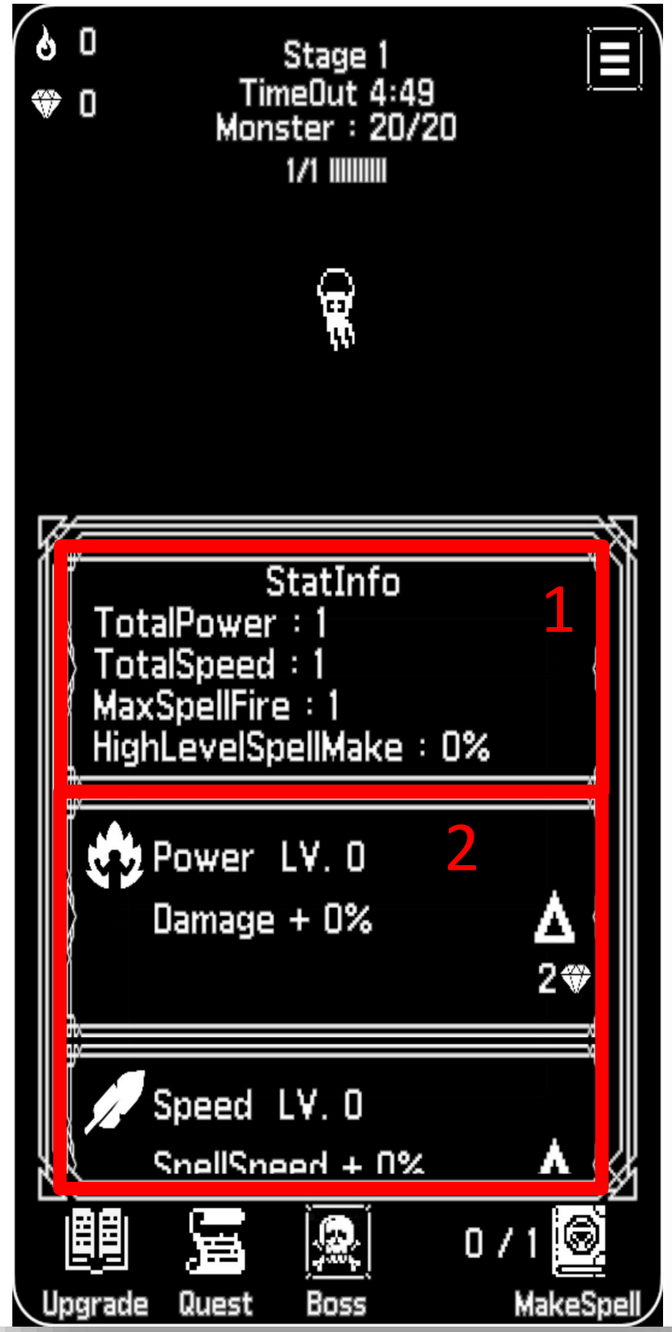
1. Info
   현재 게임의 진행 상황과 분당 획득하는 방치재화를 표시해줍니다.

2. Sound
   소리의 볼륨을 설정을 변경할수 있습니다.

3. Code
   쿠폰이나 특수 코드 입력을 위한 칸입니다.
   현재는 게임테스트를 위한 치트코드입력이 가능합니다.

4. Back 버튼
   게임화면으로 돌아가는 버튼입니다.

5. CloseGame 버튼
   게임을 종료하는 버튼입니다.



UI 설명 - 업그레이드UI

1. StatInfo
   현재 강화 되서 증가된 수치들의 총 합을 표시해줍니다.
   TotalPower = 공격력 배수
   TotalSpeed = 공격속도 배수
   MaxSpellFire = 스펠 발사대 해금 갯수
   HighLevelSpellMake = 스펠 소환 시 상위 단계 소환 확률

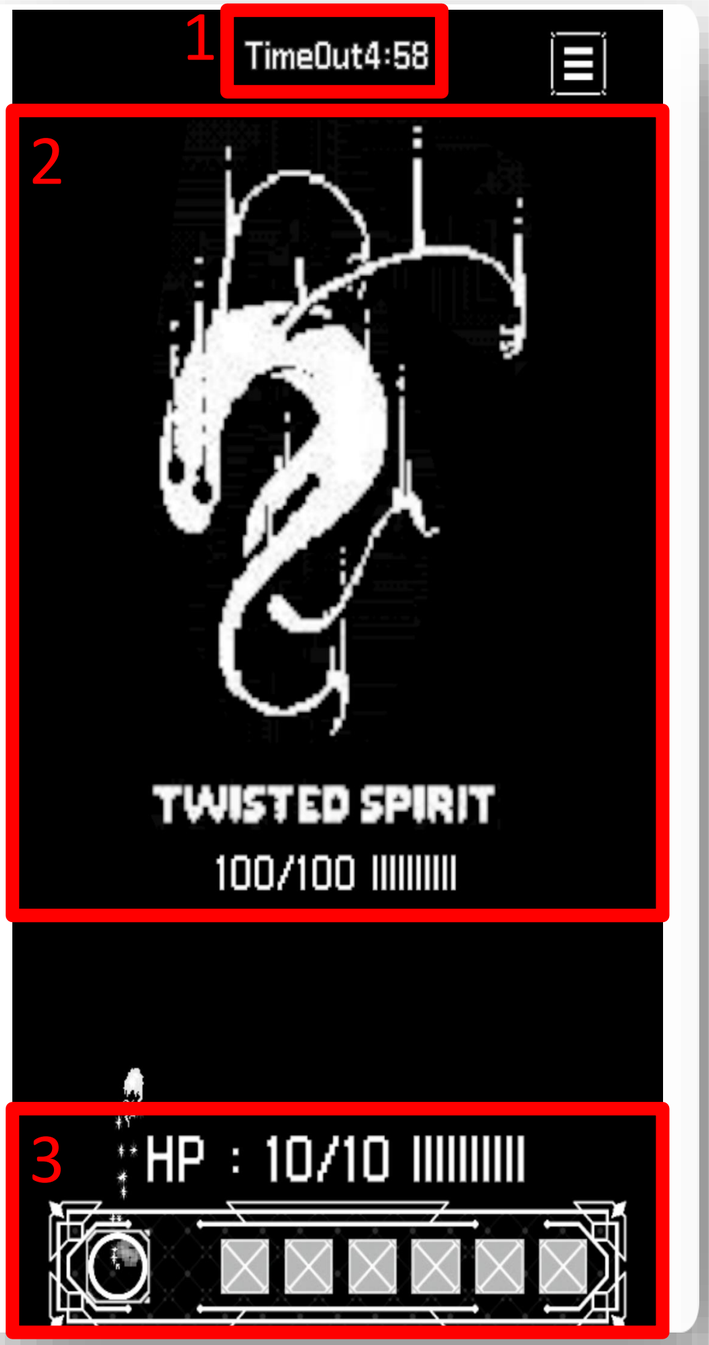
2. 업그레이드 목록
   업그레이드를 할수 있는 목록들입니다.
   왼쪽위에 아이콘과 업그레이드의 이름, 그 밑에 업그레이드 수치가 표시됩니다.
   오른쪽에 업그레이드 버튼이 있으며, 그 밑에 필요한 재화가 표시됩니다.
   스펠업그레이드의 경우 새로운 단계의 스펠을 만들 경우 해당 단계의
   스펠업그레이드가 목록에 추가됩니다.

UI 설명 - 퀘스트UI

1. 퀘스트 목록
   퀘스트를 표시해주는 목록입니다.
   왼쪽에 아이콘,퀘스트 내용이 표시되며, 그 오른쪽에 현재 달성한 횟수를 표시해줍니다.
   제일 오른쪽의 자물쇠 이미지는 퀘스트 목표가 달성되면 자물쇠가 사라지고 재화 획득 버튼이 생성되고 클릭 시 표시된 재화를 습득하고 퀘스트가 다음단계로 변경됩니다.

UI 설명 - 보스UI

1. 보스 목록
   보스를 표시해주는 목록입니다.
   왼쪽에 아이콘,보스의 이름이 표시되며, 사냥한 보스의 경우 아이콘에 체크표시가 생성됩니다.
   제일 오른쪽의 자물쇠 이미지는 이전 보스가 사냥되면 자물쇠가 사라지고 전투 아이콘이 표시가 됩니다.
   전투 아이콘을 클릭 시 해당 보스와의 전투로 화면이 전환됩니다.



UI 설명 - 보스 전투 화면

1. TimeOut
   보스 클리어의 시간제한을 표시해줍니다.

2. 보스
   맨 아래에 보스의 체력이 표시되고, 그 위에 보스의 이름, 보스의 이미지가 표시됩니다.
   보스의 체력을 0이하로 만들면 보스가 사라지면서
   “Boss Kill! You Succes”
   라는 텍스트가 나오고, 메인화면으로 돌아가서 재화를 추가해줍니다.
   플레이어의 HP가 0이하가 되거나 시간제한이 끝난 경우
   “It's your fault, Player! You Failed!”
   라는 텍스트가 나오고, 메인화면으로 돌아갑니다.

3. 플레이어
   HP라고 표시된 곳은 플레이어의 체력을 의미하며, 현재 스테이지 *10의 수치를 가집니다.
   그 밑은 스펠발사대로 보스 전투 시작 전 장착했던 스펠들을 그대로 가져옵니다.
   보스전투중에는 스펠을 교체할 수 없습니다.

03

주요 구현 기술

주요 구현 기술

스펠 소환 시스템

스펠의 이동 및 합성 시스템

스펠발사대 시스템

스펠 소환 시스템

- 스펠 소환과 리사이클링을 담당하는 MagicCircleSpawn()
- 스펠 소환 강화 정도에 따라 상위 스펠 소환 확률 계산 및 적용해주는 MakeHighLevelSpell(MagicCircle Spell)
- 스펠 소환 버튼 ClickButton()

MagicCircleMakeButton.cs

```
void MagicCircleSpawn()
{
    bool succes = false;
    for (int i = 0; i < Recycling.Count; i++)
    {
        if (!(Recycling[i].gameObject.activeSelf))
        {
            Recycling[i].transform.position = new Vector3(0, -1, -1);
            OnMagicCircle.Add(Recycling[i]);
            MagicCircle recyclingCircle = Recycling[i].GetComponent<MagicCircle>();
            recyclingCircle.SpellNum = SaveDataManager.Instance.SpellsCount;
            MakeHighLevelSpell(recyclingCircle);
            SaveDataManager.Instance.SpellsCount= OnMagicCircle.Count;
            SaveDataManager.Instance.SpellFireOn.Add(false);
            SaveDataManager.Instance.SpellLevel.Add(recyclingCircle.MagicLevel);
            Recycling[i].SetActive(true);
            succes = true;
            break;
        }
    }
    if (!succes && Recycling.Count < _maxHaveSpell)
    {
        Recycling.Add(Instantiate(MagicCirclePrefab, new Vector3(0,-1,-1), transform.rotation));
        Recycling[Recycling.Count - 1].transform.parent = CirCles.transform;
        OnMagicCircle.Add(Recycling[Recycling.Count - 1]);
        MagicCircle recyclingCircle = Recycling[Recycling.Count - 1].GetComponent<MagicCircle>();
        recyclingCircle.SpellNum = SaveDataManager.Instance.SpellsCount;
        recyclingCircle.SpellFires = SpellFireObj;
        recyclingCircle.Spawner = gameObject.GetComponent<MagicCircleMakeButton>();
        MakeHighLevelSpell(recyclingCircle);
        SaveDataManager.Instance.SpellsCount = OnMagicCircle.Count;
        SaveDataManager.Instance.SpellFireOn.Add(false);
        SaveDataManager.Instance.SpellLevel.Add(recyclingCircle.MagicLevel);
        Recycling[Recycling.Count - 1].SetActive(true);
    }
}
```

1 2 3 4 5 6 7

MagicCircleSpawn()

스펠 소환 시스템

스펠 소환과 리사이클링을 담당하는 MagicCircleSpawn() 함수

1. List<GameObject>변수 Recycling의 Count만큼 반복하면서 비활성화된 요소가 있을 시 해당 요소를 활성화 시키면서 소환
2. 소환하면서 스펠 소환 강화에 따른 상위 단계 스펠 소환 확률 적용
3. 소환 후 요소의 데이터를 세이브데이터매니저에 저장
4. bool변수 succes를 true로 바꿔주고 break를 통해 반복문 탈출
5. succes가 false이고, Rycycling의 Count가 _maxHaveSpell (스펠 최대 소유 갯수)보다 적을 시 Rycycling에 새로운 MagicCircle오브젝트 요소를 만들어서 추가해서 소환.
6. 소환하면서 스펠 소환 강화에 따른 상위 단계 스펠 소환 확률 적용
7. 소환 후 요소의 데이터를 세이브데이터매니저에 저장

MagicCircleMakeButton.cs

```
private void MakeHighLevelSpell(MagicCircle spell)
{
    int random = Random.Range(0, 100);
    if(random<SaveDataManager.Instance.MakeSpellLevel)
    {
        spell.LevelChange(spell.MagicLevel+1);
        MakeHighLevelSpell(spell);
    }
}
```

MakeHighLevelSpell(MagicCircle spell)

```
public void ClickButton()
{
    if (0 < SaveDataManager.Instance.NowMakeSpell)
    {
        if (OnMagicCircle.Count < SaveDataManager.Instance.MaxHaveSpell)
        {
            MagicCircleSpawn();
            SaveDataManager.Instance.QuestValue[10]++;//스펠제작퀘스트
            SaveDataManager.Instance.NowMakeSpell--;
            _nowMakeSpell--;
            MakeSpellUI.text = _nowMakeSpell + " / " + _maxMakeSpell;
        }
    }
}
```

ClickButton()

스펠 소환 시스템

스펠 소환 강화 정도에 따라 상위 스펠 소환 확률 계산 및 적용해주는 MakeHighLevelSpell(MagicCircle spell) 함수

- 랜덤함수로 0~100 사이의 정수 생성
- 스펠 소환 강화 레벨에 따라서 생성한 정수가 강화 레벨보다 낮을 경우
- 스펠 단계 상승
- 스펠 단계 상승 함수 재 호출 -> 더이상 단계 상승이 안되면 재호출이 종료됨

스펠 소환 버튼 ClickButton() 함수

- 세이브데이터매니저의 NowMakeSpell 변수(현재 소환 가능 횟수)가 0보다 크면서 소환 되어있는 스펠의 갯수 가 최대 소유 가능한 스펠 갯수 보다 적을 경우
- 스펠 소환 함수 호출
- 스펠 제작 퀘스트 피드백
- 현재 소환 가능 횟수 피드백
- 소환 가능 횟수에 대한 TextUI 피드백

주요 구현 기술

스펠 소환 시스템

스펠의 이동 및 합성 시스템

스펠발사대 시스템

스펠의 이동 및 합성 시스템

- 스펠을 클릭했을 때 이동과 충돌 체크를 준비하는 OnMouseDown()
- 드래그 했을 때의 이동을 담당하는 OnMouseDrag()
- 스펠의 충돌이 끝났을 때 스펠의 상태를 변경하는 OnTriggerExit2D(Collider2D collision)
- 스펠을 이동 후 마우스조작을 끝낼 때의 상태를 적용하는 OnMouseUP()

MagicCircle.cs

```
private void OnMouseDown()
{
    if (!SaveDataManager.Instance.BossFight)
    {
        gameObject.GetComponent<CircleCollider2D>().isTrigger = true;
        Effect.GetComponent<Renderer>().sortingOrder = 11;
        gameObject.GetComponent<SpriteRenderer>().sortingOrder = 11;
        Debug.Log("마법진드래그");
    }
}
```

OnMouseDown()

```
private void OnMouseDrag()
{
    if (!SaveDataManager.Instance.BossFight)
    {
        ClickPoint = Camera.main.ScreenToWorldPoint(Input.mousePosition);
        ClickPoint.z = 0;
        gameObject.transform.position = ClickPoint;
        if (!(_rigidBody2D.bodyType == RigidbodyType2D.Static))
        {
            _rigidBody2D.velocity = Vector2.zero;
        }
    }
}
```

OnMouseDrag()

스펠의 이동 및 합성 시스템

스펠을 클릭했을 때 이동과 충돌 체크를 준비하는 OnMouseDown()

- 세이브데이터매니저를 통해 보스 전투 중이 아닌지를 확인해서 아닐 경우
- 상태판별용 충돌 체크를 위한 isTrigger 활성화
- 다른 스펠보다 위에 보이기 위한 sortingOrder변경

드래그 했을 때의 이동을 담당하는 OnMouseDrag()

- 세이브데이터매니저를 통해 보스 전투 중이 아닌지를 확인해서 아닐 경우
- ClickPoint에 카메라로부터 받은 마우스 포인트의 위치를 저장하고 저장한 ClickPoint의 Z값을 조정 후 스펠의 position에 주입
- 만약 스펠의 bodyType이 스태틱이 아닐 때 velocity값을 0으로 조정

MagicCircle.cs

```
private void OnTriggerExit2D(Collider2D collision)
{
    if (collision.CompareTag("MagicCircle")
        && collision.GetComponent<MagicCircle>().MagicLevel == MagicLevel)
    {
        SameTarget = null;
    }
    if (collision.CompareTag("Wall"))
    {
        WallOut = true;
    }
    if (collision.CompareTag("SpellFire"))
    {
        OnFire = false;
    }
}
```

OnTriggerExit2D(Collider2D collisoin)

스펠의 이동 및 합성 시스템

스펠의 충돌이 끝났을 때 스펠의 상태를 변경하는
OnTriggerExit2D(Collider2D collisoin)

1. 충돌이 끝난 오브젝트의 태그가 “MagicCircle”이면서 해당 오브젝트와 MagicLevel의 값이 같을 경우
2. 같은 단계의 오브젝트를 저장하는 SameTarget 을 null로 변경
3. 충돌이 끝난 오브젝트의 태그가 “Wall”일 경우
4. 스펠저장소 밖을 나갔는지 판별하는 상태인 WallOut을 true로 변경
5. 충돌이 끝난 오브젝트의 태그가 “SpellFire”인 경우
6. 스펠발사대 위에 올라갔다는 상태를 의미하는 OnFire를 false로 변경

MagicCircle.cs

```
private void OnMouseUp()
{
    if (!SaveDataManager.Instance.BossFight)
    {
        if (OnFire
            && SpellFires.FireSpellList.Count < SaveDataManager.Instance.MaxSpellFiresCount)
        {
            SameTarget = null;
            SpellFires.OnSpell(gameObject);
            SaveDataManager.Instance.SpellFireOn[SpellNum] = true;
            Effect.GetComponent<Renderer>().sortingOrder = 11;
            gameObject.GetComponent<SpriteRenderer>().sortingOrder = 11;
            gameObject.GetComponent<CircleCollider2D>().isTrigger = true;
        }
        else
        {
            if (SameTarget != null)
            {
                if (SameTarget.activeSelf
                    && SameTarget.GetComponent<MagicCircle>().MagicLevel == MagicLevel
                    && MagicLevel < 11 && !OnFire)
                {
                    SameTarget.SetActive(false);
                    LevelChange(MagicLevel + 1);
                    SaveDataManager.Instance.SpellLevel[SpellNum] = MagicLevel;
                    SaveDataManager.Instance.QuestValue[11]++;//스펠합성퀘스트
                }
                SameTarget = null;
            }
            if (SpellFires != null)
            {
                SpellFires.OffSpell(gameObject);
                SaveDataManager.Instance.SpellFireOn[SpellNum] = false;
            }
            if (WallOut)
            {
                transform.position = new Vector2(0, -1);
                _rigidBody2D.AddForce(new Vector2(Random.Range(-1, 1), Random.Range(-1, 1)));
                WallOut = false;
            }
            Effect.GetComponent<Renderer>().sortingOrder = 9;
            gameObject.GetComponent<SpriteRenderer>().sortingOrder = 9;
            gameObject.GetComponent<CircleCollider2D>().isTrigger = false;
        }
    }

}
```

OnTriggerExit2D(Collider2D collisoin)

스펠의 이동 및 합성 시스템

스펠의 충돌이 끝났을 때 스펠의 상태를 변경하는 OnTriggerExit2D(Collider2D collisoin)

1. 세이브데이터매니저를 통해 보스 전투 중이 아닌지를 확인해서 아닐 경우
   ① 스펠발사대 위에 올라간 상태라는 OnFire가 true이면서 스펠 발사대의 자리가 남았을 경우
      A. 같은 단계의 충돌한 스펠을 저장하는 SameTarget을 null로 변경
      B. 스펠발사대에 스펠을 저장하고, 그 내용을 세이브데이터매니저에 피드백
   ② 스펠 발사대 위가 아니거나 스펠 발사대에 자리가 없을 경우
      A. 같은 단계의 충돌한 스펠(SameTarget)이 있을 경우
         a. SameTarget이 활성화 상태면서 현재 MagicLevel이 같고, MagicLevel이 11을 넘지 않으면서 OnFire가 false일 경우
            - SameTarget에 저장된 오브젝트를 비활성화
            - 본 스펠의 단계를 1 상승
            - 스펠의 단계와 퀘스트 내용을 세이브데이터매니저에 피드백
         b. SameTarget을 null로 변경
      B. SpellFires가 null이 아닐때
         a. 스펠발사대에서 본 스펠을 제거하고 세이브데이터매니저에 피드백
      C. 스펠이 스펠 저장소 밖으로 나갔을 때(WallOut==true)
         a. 스펠의 위치를 스펠저장소 안으로 초기화하고, 다른 스펠과 완전히 겹치지 않게 랜덤으로 짧게 이동
         b. WallOut을 false로 변경
      D. 트리거 충돌 체크 비활성을 위해 isTrigger를 false로 변경

주요 구현 기술

스펠 소환 시스템

스펠의 이동 및 합성 시스템

스펠발사대 시스템

스펠의 이동 및 합성 시스템

- 스펠발사대에 스펠을 추가하는 OnSpell(GameObject spell)
- 스펠발사대에서 스펠을 제거하는 OffSpell(GameObject spell)

- 스펠발사대의 목록에 있는 스펠을 발동하는 FireSpell()

MagicCircle.cs

```
public void OnSpell(GameObject spell)//스펠을 발동 목록에 추가
{
    bool same = false;
    for (int i = 0; i < FireSpellList.Count; i++)
    {
        if (FireSpellList[i] == spell)
        {
            same = true;
        }
    }
    if (!same)
    {
        FireSpellList.Add(spell);
        spell.transform.parent = FireSpellParent.transform;
    }
    SpellReposition();

}
```

OnSpell(GameObject spell)

```
public void OffSpell(GameObject spell)//스펠을 발동 목록에서 제거
{
    for(int i=0;i<FireSpellList.Count;i++)
    {
        if(FireSpellList[i]==spell)
        {
            FireSpellList.RemoveAt(i);
            spell.transform.parent = SpellTankParent.transform;
            spell.GetComponent<Rigidbody2D>().bodyType = RigidbodyType2D.Dynamic;
        }
    }
    SpellReposition();

}
```

OffSpell(GameObject spell)

스펠의 이동 및 합성 시스템

스펠발사대에 스펠을 추가하는 OnSpell(GameObject spell)

1. 스펠 발사대 목록에서 지정 받은 스펠이 이미 있는지 확인
2. 발사대 목록에 스펠이 없다면 목록에 스펠을 추가하고, 스펠의 부모 오브젝트를 스펠발사대로 변경
3. 스펠발사대 목록의 스펠들의 위치를 재정렬

스펠발사대에서 스펠을 제거하는 OffSpell(GameObject spell)

1. 스펠 발사대 목록에서 지정 받은 스펠이 있는지 확인
2. 목록에 해당 스펠이 존재한다면 목록에서 해당 스펠을 제거
3. 해당 스펠의 부모 오브젝트를 스펠 저장소로 변경
4. 해당 스펠의 바디타입을 다이나믹으로 변경
5. 스펠발사대 목록의 스펠들의 위치를 재정렬

MagicCircle.cs

```
public void FireSpell()//목록에 있는 스펠을 발동
{
    int SpellTimerInt = (int)SpellTimer/1;
    if(SpellTimerInt != NowSpellNum && SpellTimerInt < 8)
    {
        SpellFireSound.SetActive(false);
        NowSpellNum = SpellTimerInt;
        if(NowSpellNum<FireSpellList.Count)
        {
            SpellFireSound.SetActive(true);
            int magicLevel = FireSpellList[NowSpellNum].GetComponent<MagicCircle>().MagicLevel;
            Vector3 spellPosition = FireSpellList[NowSpellNum].transform.position;
            GameObject fireSpell =
                Instantiate(SpellEffects[magicLevel], spellPosition , Quaternion.identity);
            float damage =
                SaveDataManager.Instance.BaseDamage[magicLevel]
                * SaveDataManager.Instance.TotalPower
                * (1+SaveDataManager.Instance.SpellPower[magicLevel]*0.1f);
            fireSpell.GetComponent<SpellData>().SetData(target,damage);
        }
        if(1<=NowSpellNum)
        {
            NowSpellLine[NowSpellNum - 1].SetActive(false);
        }
        NowSpellLine[NowSpellNum].SetActive(true);
    }
    if (MaxSpellFireLine <= SpellTimer)
    {
        NowSpellLine[MaxSpellFireLine-1].SetActive(false);
        SpellTimer = 0;
        NowSpellNum = -1;
        Fire = false;
        StartCoroutine(FireTimer());
    }
    else
        SpellTimer += SaveDataManager.Instance.TotalSpeed * Time.deltaTime;
}
```

FireSpell()

스펠의 이동 및 합성 시스템

스펠발사대의 목록에 있는 스펠을 발동하는 FireSpell()

1. SpellTimerInt에 SpellTimer (스펠발사타이밍)의 정수 추출

2. SpellTimverInt가 NowSpellNum (지금 발동중인 스펠의 번호) 과 다르고, 8보다 작을때
   A. NowSpellNum이 스펠발사대 목록의 최댓값보다 작은경우
      a. 스펠 발사 사운드 출력
      b. 스펠의 레벨과 발사대에서의 위치를 받아서 스펠의 투사체 소환
      c. 스펠의 단계에 따른 베이스 데미지와 총합 강화 파워, 해당 스펠 업그레이드의 레벨에 따른 데미지 공식을 계산하여서 스펠투사체의 데이터에 적용
   B. NowSpellNum이 1보다 크거나 같을때
      a. NowSpellNum-1 번째의 스펠 발동 표시 비활성화
   C. NowSpellNum번째 스펠의 발동 표시 활성화

3. MaxSpellFireLine (스펠발사대 최대길이)보다 SpellTimer가 크거나 같을경우
   A. 마지막 스펠 발동 표시 비활성화
   B. SpellTimer 를 0으로 변경
   C. NowSpellNum 을 -1로 변경
   D. 스펠 발동 상태를 의미하는 Fire를 false로 변경
   E. FireTimer() 작동 (1초후 스펠 발사대를 재발동 하는 함수)

4. MaxSpellFireLine보다 SpellTimer가 크거나 같을경우가 아닐때
   A. SpellTimer에 스펠발사속도를 적용

Thank You!

감사합니다!